1.1 Introduction & Motivation

In this notebook, we will discuss the following points

- Why find the minimum (or maximum) of a function?
- Why Julia?
- Why Scientific Machine Learning?

Import package to plot graphs in Julia

In [1]:
```
using CairoMakie
```

Why do we need to find the minimum (or maximum) of a function?

There are many engineering applications where we need to find the minimum of a function. For example

- Minimize the material needed to build a computer to reduce weight and cost.
- Minimum thickness of a beam so that it can support a load on a building
- Minimizing time to transport assets/goods to a particular destination to save cost.
- In machine learning, you want to minimize the cost function so that you can get the best possible mathematical model for accurate predictions of an engineering system.

Exercise Ex01: You want to make a cylinder that can store up to 1 m ^{3} of water.

- Show that the area of the cylinder is given by $A(r) = 2\pi r^2 + \frac{2}{r}$
- What is the radius r such that you minimize the surface area of the cylinder (so that you need the least amount of material to make this cylinder)?

Let's plot $A(r)$ to see how it looks like

In [6]:
```
A(r)=2π*r^2+2/r # Define function A(r)
rplot=0.1:0.01:2 #define the range of the plot
fig = Figure() #set up a figure
ax = Axis(fig[1, 1], xlabel = L"r", ylabel = L"A(r)") #Define axis in the
lines!(ax,rplot,A.(rplot);color=:black) #Plot line
fig #show the figure
save("../figures/01p01IntroductionAndMotivation01.svg", fig) #save figure
```

```
CairoMakie.Screen{SVG}
```

As you can see from the graph above, the area of the material needed to make this cylinder $A(r)$ has a minimum value of around $r \approx 0.5$ m, which has a corresonding value of $A \approx 6$ m ^{2}. You would like to make your cylinder at around $r \approx 0.5$ m to ensure you use the least material possible.

Exercise Ex02: Show that the minimum of $A(r)$ occurs at $r = \sqrt[3]{\frac{1}{2\pi}}$

Why Julia?

There are many programming languages out there. Amongst the machine learning community, Python is probably the most widely used because it is very easy to learn and has many readily available packages for machine learning. However, Python is also quite slow because it was not originally designed to be for machine learning tasks.

Julia, on the other hand is quite a new language designed to be fast. See for example the benchmark below taken from

https://julialang.org/benchmarks/

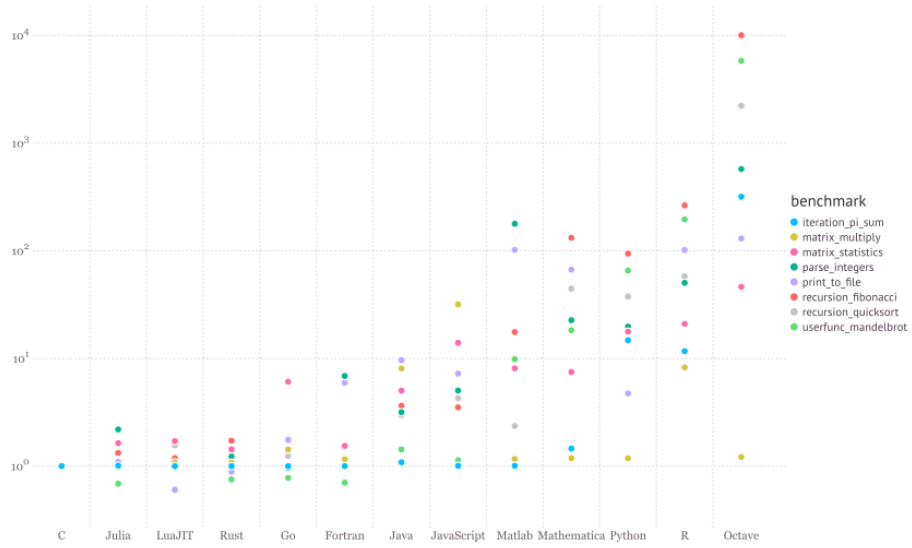


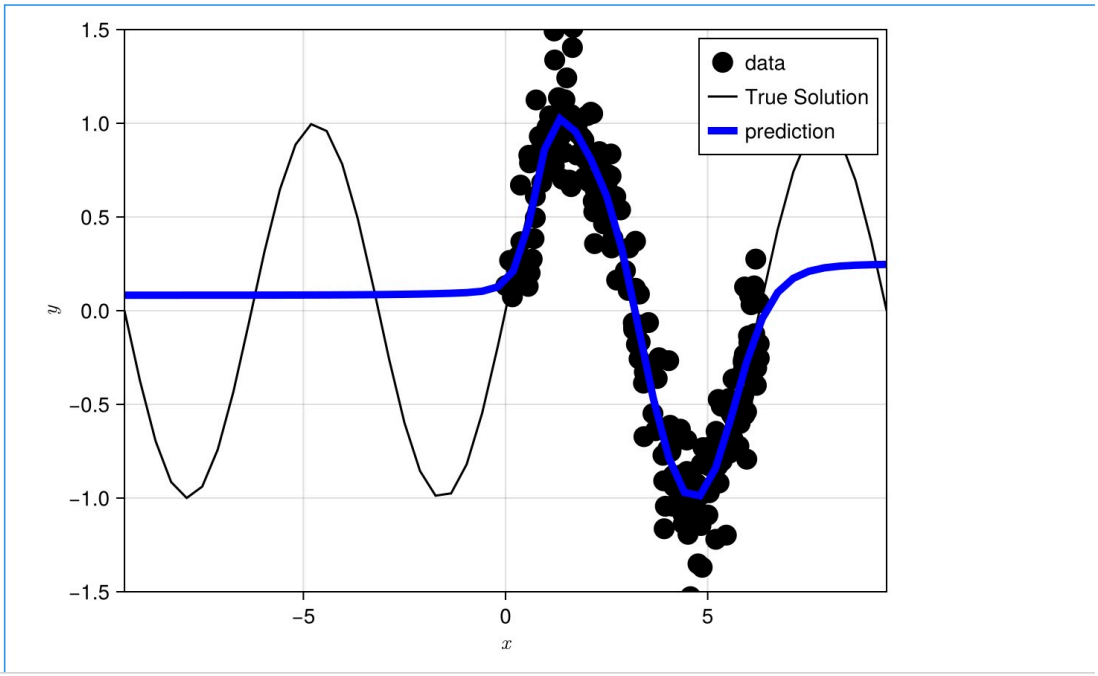
Figure 1: Julia Benchmark.

Why Scientific Machine Learning?

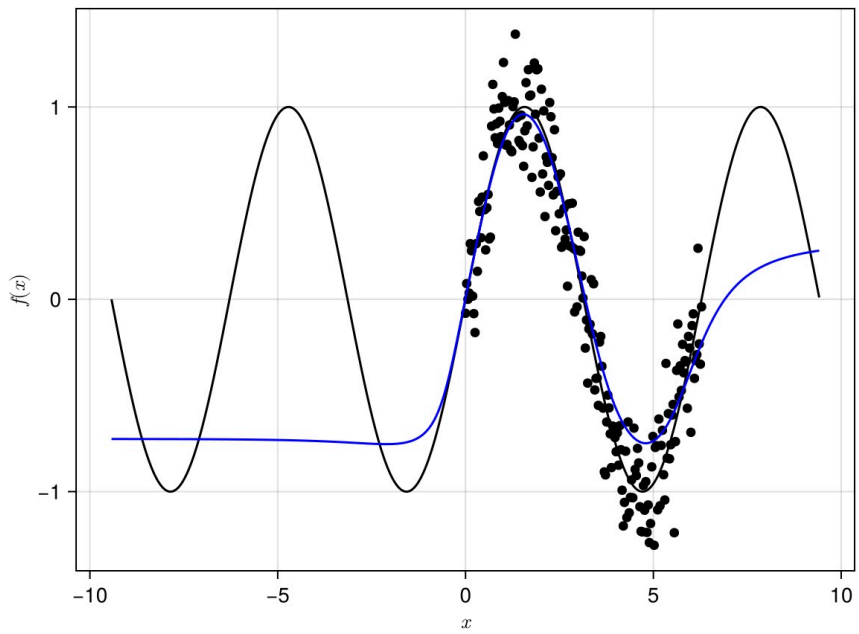
In traditional machine learning, predictions are made using mathematical models calibrated purely based on data. This approach is reasonable when you have lots of data (such as image recognition, large language models etc). However, for scientific problems, it is not often that you have lots of data. It is more common that scientific data is scarce. Furthermore, high quality data is difficult and very costly to obtain. Hence, in order to obtain a good predictions, we need to integrate/blend machine learning models with domain specific knowledge based on fundamental scientific principles and conservation laws.

The figures below show the benefits of scientifc machine learning. The first image below shows that with traditional machine learning methods, the prediction of the

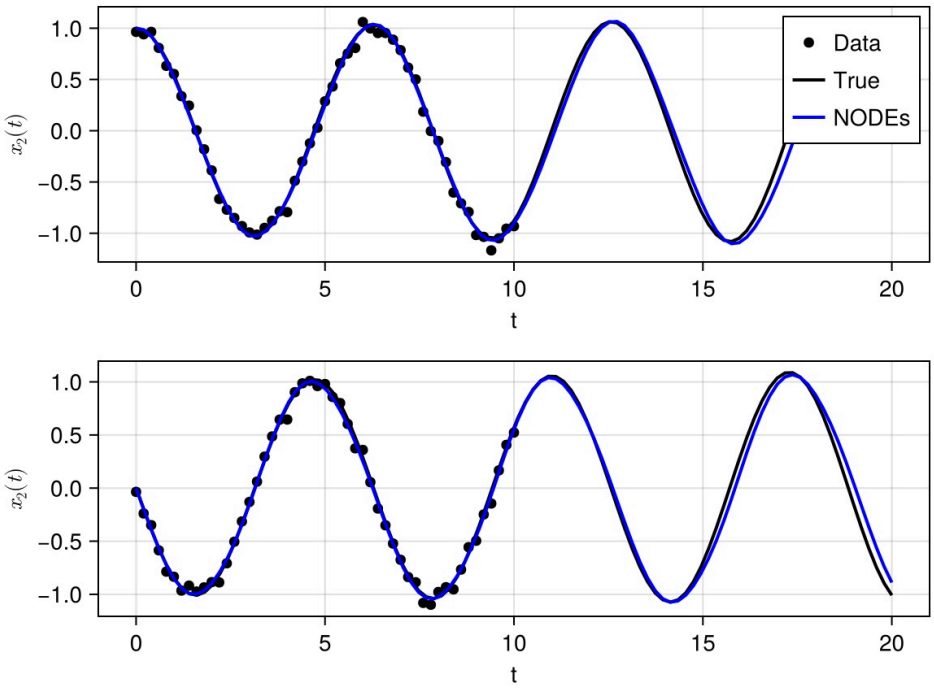
model is good within the regions when you have data. However, if you try to use the model to predict into areas where you do not have data, the prediction is usually bad.



The image below incorporates a bit of scientific knowledge into the prediction. As you can see, the predictions is usually better but still not very good, especially when you get further away from the data set.



The results below reformulates the traditional machine learning approches and uses Neural Network in a different manner such that it incorporates traditional scientific principles. As you can see, you can get much btter predictions, even far away from where the data is collected.



- Prepared by Andrew Ooi 2nd July 2026